

Lectures on Monte Carlo Theory

Chapter 1: Introduction to Monte Carlo

Based on Leskälä & Vihola

Chapter Outline

- 1 1.1 About the Book
- 2 1.2 Simple Symmetric Random Walk
- 3 1.3 Travelling Salesman Problem
- 4 1.4 Computing Integrals with Monte Carlo
- 5 1.5 Quasi-Monte Carlo Methods
- 6 1.6 MC vs. Quasi-MC: Comparison
- 7 1.7 Bibliographical Notes

1.1 What Are Monte Carlo Methods? I

Definition — Monte Carlo (MC) Methods

Monte Carlo methods are computational algorithms that rely on *repeated random (pseudo-random) sampling* to obtain numerical results. They are used to solve problems in numerical integration, optimisation, simulation of stochastic systems, and Bayesian statistics.

The name “Monte Carlo” was coined by Metropolis and Ulam in 1949, after the famous casino in Monaco, because the method resembles repeated gambling experiments. The approach was developed at Los Alamos in the 1940s by Stanislaw Ulam, John von Neumann, and Nicholas Metropolis, originally to simulate neutron diffusion in nuclear weapons design.

The fundamental idea is elegant: if you cannot compute a quantity analytically (by formula), *simulate* the underlying random experiment a very large number of times and average the results. Two classical theorems from probability theory guarantee that this works:

- The **Law of Large Numbers** (LLN) guarantees that the average of many independent repetitions converges to the true expected value — the method is correct in the limit.

1.1 What Are Monte Carlo Methods? II

- The **Central Limit Theorem** (CLT) tells us how large the error is for a given number of repetitions, allowing us to build confidence intervals around our estimate.

1.1 What Are Monte Carlo Methods? III

Why does randomness help? There are three broad problem classes where deterministic methods struggle and Monte Carlo excels:

- 1 **High-dimensional integration.** Evaluating an integral over a d -dimensional domain using a regular grid requires m^d function evaluations (where m is the number of grid points per dimension). This is called the *curse of dimensionality* — the cost grows exponentially with d . For example, with $m = 10$ points per dimension and $d = 20$ dimensions, a grid would require 10^{20} evaluations — far beyond any computer. The Monte Carlo estimator, by contrast, achieves a root-mean-squared error of $O(n^{-1/2})$ regardless of d , where n (*italic n*) is the number of random samples. This *dimension-independence* makes MC the method of choice for high-dimensional problems in finance, physics, and machine learning.
- 2 **NP-hard combinatorial problems.** Problems like the Travelling Salesman Problem (TSP) have a search space of $n!$ (read “ n factorial”) permutations, which grows faster than any exponential. For $n = 50$ cities, $50!$ is approximately 3×10^{64} — exhaustive search is physically impossible. Monte Carlo heuristics such as simulated annealing and the cross-entropy method find near-optimal solutions efficiently by intelligently exploring the space.

1.1 What Are Monte Carlo Methods? IV

- ③ **Complex stochastic systems.** When the dynamics of a system involve intrinsic randomness (e.g., queuing networks, financial models, particle physics), direct simulation is often the only tractable approach. We simply run the system many times and record statistics of interest.

1.1 What Are Monte Carlo Methods? V

Structure of this book. The book is structured to build from foundational theory to advanced methods, always connecting mathematical theory to Python implementations. Chapter 1 provides three motivating examples that recur throughout the text.

- ① **Ch. 1** Introduction: three motivating examples — random walks (the arcsine law and the law of the iterated logarithm), the Travelling Salesman Problem, and numerical integration.
- ② **Ch. 2** Theory of pseudorandom number generators (PRNGs) and statistical tests for randomness quality.
- ③ **Ch. 3** Generating random variables from arbitrary distributions: inversion, rejection, and transformation methods.
- ④ **Ch. 4** Output analysis: confidence intervals, bias, variance estimation, and consistent estimators.
- ⑤ **Ch. 5** Variance reduction techniques: stratification, antithetic variables, importance sampling, and the cross-entropy method.
- ⑥ **Ch. 6** Markov Chain Monte Carlo (MCMC): the Metropolis–Hastings algorithm, the Gibbs sampler, and coupling from the past (CFTP) for exact sampling.

1.1 What Are Monte Carlo Methods? VI

- 7 **Ch. 7** Stochastic optimisation: simulated annealing and the cross-entropy method applied to the TSP and other problems.
- 8 **Ch. 8** Simulation of queues and related stochastic processes.

Key Distinction: MC vs. Quasi-MC

Monte Carlo (MC) uses *pseudo-random* sequences — deterministic but statistically indistinguishable from true randomness. **Quasi-Monte Carlo (QMC)** uses carefully designed *low-discrepancy* sequences that fill $[0, 1]^d$ (the d -dimensional unit hypercube) more uniformly than random points. QMC can converge faster for smooth integrands, but has important limitations for simulation tasks. Section 1.4 and 1.5 compare the two approaches in detail.

1.1 Pseudorandom Numbers in Python

Before tackling the theory, it is essential to know how to generate pseudo-random numbers in Python. The NumPy library provides a modern, reproducible interface via `default_rng()`. Passing a seed (an integer starting value) makes the entire sequence deterministic, which is crucial for reproducibility and debugging: if you share your seed, anyone can reproduce your results exactly.

NumPy Recommended API (NumPy $\geq$ 1.17)

The function `default_rng(seed)` returns a generator object backed by the PCG64 algorithm, where `seed` is any integer you choose. Calling `rng.random(k)` draws k (integer k) samples from $\mathcal{U}[0, 1)$ (the uniform distribution on the interval from 0 inclusive to 1 exclusive). Calling `rng.integers(low=0, high=M, size=k)` draws k integers uniformly from $\{0, 1, \dots, M - 1\}$.

```
import numpy as np
from numpy.random import default_rng, PCG64

# Recommended: default_rng with explicit seed
rng = default_rng(seed=31415)           # PCG64 generator, seeded

u = rng.random(size=5)                  # 5 samples from Uniform[0,1)
b = rng.integers(low=0, high=2, size=10) # 10 values in {0,1}
steps = 1 - 2 * b                       # convert {0,1} -> {-1,+1}

print("Uniform samples:", u)
```

1.2.1 Setup: The Coin-Toss Game I

The *simple symmetric random walk* is one of the most fundamental objects in probability theory. It models phenomena as diverse as stock price fluctuations, polymer chain conformations, and diffusion of particles in a liquid. It is a perfect test bed for Monte Carlo simulation: the exact analytical answers are known from classical limit theorems (due to Feller, 1950), so we can verify that our simulations converge to the right values. This is how practitioners “validate” simulation code before applying it to problems with unknown answers.

The game (Feller, 1950). Two players, A and B, repeatedly bet on coin tosses. Player A wins +1 monetary unit from B when heads appears, and loses 1 unit when tails appears. Player A's cumulative fortune (net winnings) after n (integer n) tosses is denoted S_n (S subscript n). The starting fortune is $S_0 = 0$.

1.2.1 Setup: The Coin-Toss Game II

Definition — Simple Symmetric Random Walk (SSRW)

Let $X_1, X_2, \dots$ (X subscript 1, X subscript 2, etc.) be independent and identically distributed (i.i.d.) random variables with

$$\mathbb{P}(X_j = +1) = \mathbb{P}(X_j = -1) = \frac{1}{2}.$$

Here $\mathbb{P}$ (blackboard P) denotes probability, $X_j = +1$ means heads on toss j , and $X_j = -1$ means tails. The symbol j (j) runs over $1, 2, 3, \dots$. Define the **partial sums**:

$$S_0 = 0, \quad S_n = \sum_{j=1}^n X_j, \quad n = 1, 2, \dots$$

Here $\sum$ (Sigma, meaning “sum”) adds up all the steps X_j for j running from 1 to n . The sequence $S_0, S_1, S_2, \dots$ is called the **simple symmetric random walk**.

1.2.1 Setup: The Coin-Toss Game III

How to Read This Formula

S_n is player A's running total after n coin tosses. Because the steps X_j are independent (each toss is fair), the walk is equally likely to go up or down at each step. Note that S_n must lie in the set $\{-n, -n+2, \dots, n-2, n\}$, since the net result of n steps of ± 1 has the same parity as n . In particular, $|S_n| \leq n$: the walk can never exceed its maximum possible value.

1.2.1 Setup: The Coin-Toss Game IV

Four questions about the walk. We consider a walk of $2N$ steps (an even number, so that ties can only occur at even times). The following four questions all have surprising answers:

- Q1. Let L_{2N} (L subscript $2N$) $= \max\{i \leq 2N : S_i = 0\}$ be the *last moment of a tie*, i.e., the last time the two players are exactly equal. What is the distribution of L_{2N} ? Intuition says it should be near the end — is it?
- Q2. Let $P(\alpha, \beta)$ (where α (alpha) and β (beta) are fractions between 0 and 1 with $\alpha < \beta$) be the probability that player A is winning for a fraction of time between α and β . Is this distribution concentrated near $1/2$, as one might expect for a fair game?
- Q3. How do the oscillations (fluctuations) of S_n grow with n ? Are they proportional to n , to $\sqrt{n}$ (square root of n), or something in between?
- Q4. What is the probability that one player is *always* winning, i.e., that $S_k > 0$ for all steps $k = 1, \dots, 2N$?

All four questions have exact analytical answers from classical limit theorems. Monte Carlo simulation lets us *verify* these answers numerically and develop intuition about them before studying the proofs.

1.2.1 Setup: The Coin-Toss Game V

Formal definitions of the key quantities.

Last Tie (L_{2n}) and Time Above Zero (L_{2n}^+)

$$L_{2n} = \sup\{m \leq 2n : S_m = 0\}$$

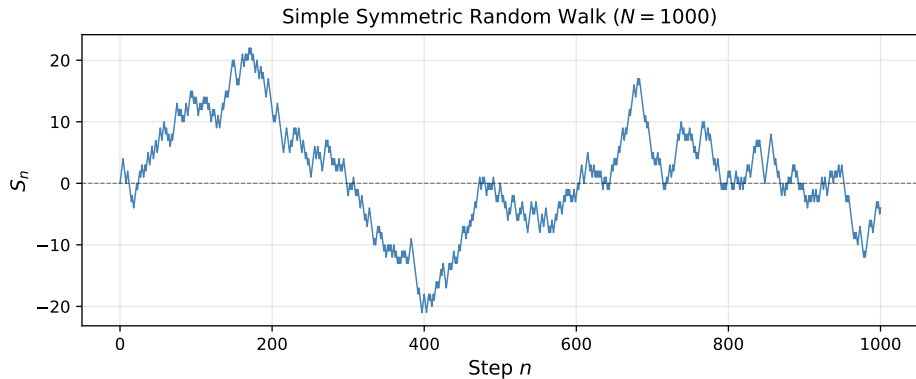
reads as: L_{2n} is the *last time up to step $2n$ at which the walk equals zero*, i.e., the last tie. Here “sup” means *supremum* (the largest element of the set).

$$L_{2n}^+ = \#\{j = 1, \dots, 2n : S_{j-1} \geq 0 \text{ or } S_j \geq 0\}$$

reads as: L_{2n}^+ (L-plus subscript $2n$) is the *number of steps during which player A is winning*. A step j is counted as “above zero” if *at least one* of the two endpoints of the segment (from step $j - 1$ to step j) is non-negative. The symbol $\#$ (hash or “counting measure”) denotes the number of elements in a set.

Both L_{2n} and L_{2n}^+ grow to infinity as $n \rightarrow \infty$ (as the number of steps n grows without bound). However, the *fractions* $L_{2n}/(2n)$ and $L_{2n}^+/(2n)$ both lie in $[0, 1]$, and their distributions converge to the famous *arcsine distribution*.

1.2.1 Setup: The Coin-Toss Game VI



A single realisation of the SSRW for $N = 1000$ steps. Notice the long “excursions” on one side of zero — a consequence of the arcsine law discussed in Section 1.2.3.

1.2.2 How Large Are the Fluctuations? The Law of the Iterated Logarithm

Our first question is: how large does $|S_n|$ (the absolute value of the walk, measuring how far from zero it has gone) get as n grows?

Three candidate growth rates. Since $|S_n| \leq n$, one might try several normalisations:

- **Normalise by n :** The strong law of large numbers (SLLN) gives $S_n/n \rightarrow 0$ almost surely (with probability 1) because the mean of each step is zero. This means dividing by n is *too aggressive* — the normalised walk vanishes.
- **Normalise by $\sqrt{n}$ (square root of n):** The Central Limit Theorem (CLT) says $S_n/\sqrt{n}$ converges *in distribution* to $\mathcal{N}(0, 1)$ (a standard normal distribution, the bell curve). Here $\mathcal{N}(0, 1)$ has mean 0 and variance 1. However, the sequence $S_n/\sqrt{n}$ does *not* converge almost surely — it oscillates indefinitely. So $\sqrt{n}$ is *too weak*: the normalised walk does not settle down.
- **Normalise by $\sqrt{2n \log \log n}$:** This is the *correct* scale, as stated by the Law of the Iterated Logarithm. Here “log” (log) means the natural logarithm (base $e \approx 2.718$), and “log log n ” means the logarithm of the logarithm — a very slowly growing function.

1.2.2 How Large Are the Fluctuations? The Law of the Iterated Logarithm II

Law of the Iterated Logarithm (LIL) — Khintchine, 1924; Kolmogorov, 1929

For the simple symmetric random walk $S_n = \sum_{j=1}^n X_j$, where $X_j \in \{-1, +1\}$ each with probability $1/2$:

$$\mathbb{P}\left(\limsup_{n \rightarrow \infty} \frac{S_n}{\sqrt{2n \log \log n}} = +1\right) = 1 \quad \text{and} \quad \mathbb{P}\left(\liminf_{n \rightarrow \infty} \frac{S_n}{\sqrt{2n \log \log n}} = -1\right) = 1.$$

Here “lim sup” (limit superior, or lim-sup) is the largest accumulation point of the sequence — the highest value it gets close to infinitely often. Symmetrically, “lim inf” (limit inferior) is the smallest accumulation point. The formula says both are ± 1 with probability 1.

1.2.2 How Large Are the Fluctuations? The Law of the Iterated Logarithm III

How to Read the LIL

In plain language: the walk S_n fluctuates without ever settling down. It touches values arbitrarily close to $+\sqrt{2n \log \log n}$ (the upper envelope) and $-\sqrt{2n \log \log n}$ (the lower envelope) infinitely often as n grows. But it never permanently *exceeds* these envelopes. The “iterated logarithm” $\log \log n$ is the correction factor beyond $\sqrt{n}$: it grows incredibly slowly (for $n = 10^{100}$, we have $\log \log n \approx 5.4$), so the envelope is only slightly wider than $\sqrt{n}$.

1.2.2 How Large Are the Fluctuations? The Law of the Iterated Logarithm IV

What does $\sqrt{2n \log \log n}$ look like numerically?

Let us plug in a concrete value. For $n = 10,000$:

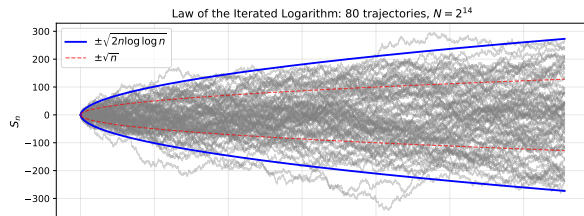
$$\sqrt{n} = \sqrt{10,000} = 100,$$

$$\log n = \log(10,000) \approx 9.21 \quad (\text{natural log}),$$

$$\log \log n = \log(9.21) \approx 2.22,$$

$$\sqrt{2n \log \log n} = \sqrt{2 \times 10,000 \times 2.22} \approx \sqrt{44,400} \approx 211.$$

So at step $n = 10,000$, the LIL envelope is approximately 211, which is about 2.1 times larger than $\sqrt{n} = 100$. The gap between the two envelopes widens only very slowly (logarithmically) as n grows.



Simulation evidence. The figure shows 80 independent SSRW trajectories of length $n = 2^{14} = 16,384$ steps. The blue curves are the LIL envelopes $\pm\sqrt{2n \log \log n}$; the red dashed lines are $\pm\sqrt{n}$ for comparison. Every simulated path stays inside the LIL envelope, consistent with the theorem. The paths

1.2.2 How Large Are the Fluctuations? The Law of the Iterated Logarithm

Key Takeaway

The walk grows neither as slowly as $\sqrt{n}$ nor as fast as n . The correct scale is $\sqrt{2n \log \log n}$, which is slightly (but only slightly) larger than $\sqrt{n}$. This precise scaling has important consequences for statistical tests and for understanding variability in long simulations.

1.2.3 The Arcsine Law I

The arcsine law answers questions Q1 and Q2, and is among the most counter-intuitive results in elementary probability theory.

What you might expect. Since the walk is perfectly symmetric (equal probability of going up or down), you might think that the fraction of time that player A is winning is typically close to $1/2$. After all, the game is fair, so shouldn't both players lead for roughly equal amounts of time? The arcsine law says the *opposite*: being ahead for *nearly all* or *nearly none* of the time is *most likely*, and being ahead for exactly half the time is the *least likely* outcome.

Arcsine Law (Feller, 1950; Durrett, 1996)

For $0 \leq \alpha < \beta \leq 1$ (fractions between 0 and 1), as $n \rightarrow \infty$ (as the number of steps grows without bound):

$$\mathbb{P}\left(\alpha \leq \frac{L_{2n}^+}{2n} \leq \beta\right) \longrightarrow \int_{\alpha}^{\beta} \frac{dx}{\pi \sqrt{x(1-x)}}. \quad (1.2.1)$$

1.2.3 The Arcsine Law II

Here π (pi) is the mathematical constant ≈ 3.14159 , and the integral $\int_{\alpha}^{\beta}$ (integral from α to β) computes the area under the curve $1/(\pi\sqrt{x(1-x)})$ between $x = \alpha$ and $x = \beta$. The same limiting distribution holds for $L_{2n}/(2n)$.

Why is this called the “arcsine” law? The cumulative distribution function (the probability that the fraction is at most β) is:

$$\int_0^{\beta} \frac{dx}{\pi\sqrt{x(1-x)}} = \frac{2}{\pi} \arcsin(\sqrt{\beta}),$$

where $\arcsin$ (arcsine, the inverse of the sine function) appears. This is where the name comes from. The probability density function $p(x) = 1/(\pi\sqrt{x(1-x)})$ has a characteristic **U-shape**: it equals $+\infty$ at $x = 0$ and $x = 1$ (the walk is most likely to spend all or none of its time above zero), and achieves its *minimum* value of $2/\pi \approx 0.637$ at $x = 1/2$ (the least likely outcome is splitting the lead evenly).

1.2.3 The Arcsine Law III

Numerical illustration of the U-shape. Let us compute the probability of a “fair” outcome (player A leads between 40% and 60% of the time) and compare it to the probability of an “extreme” outcome (player A leads more than 90% or less than 10% of the time):

1.2.3 The Arcsine Law IV

Worked Numerical Calculation

The probability of a “near-equal” outcome:

$$\mathbb{P}\left(0.4 \leq \frac{L_{2n}^+}{2n} \leq 0.6\right) \approx \frac{2}{\pi} \arcsin(\sqrt{0.6}) - \frac{2}{\pi} \arcsin(\sqrt{0.4}) \approx 0.795 - 0.667 \approx 0.128.$$

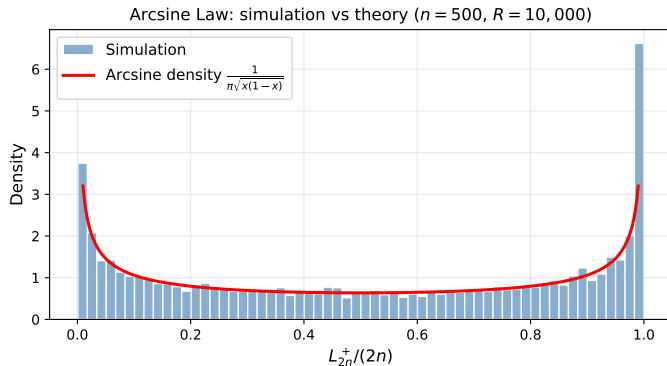
Only about 12.8% of games end with a “fair” lead split!

The probability of an “extreme” outcome (one player dominates):

$$\mathbb{P}\left(\frac{L_{2n}^+}{2n} \leq 0.1\right) + \mathbb{P}\left(\frac{L_{2n}^+}{2n} \geq 0.9\right) \approx 2 \times \frac{2}{\pi} \arcsin(\sqrt{0.1}) \approx 2 \times 0.205 \approx 0.410.$$

Over 41% of games end with one player leading for more than 90% of the time — more than three times as common as the “fair” outcome!

1.2.3 The Arcsine Law V



Histogram of $L_{1000}^+/1000$ from $R = 10,000$ replications of 1000-step walks. The red curve is the theoretical arcsine density $1/(\pi\sqrt{x(1-x)})$. The U-shape is clearly visible: most walks spend nearly all their time on one side of zero.

1.2.3 The Arcsine Law VI

Key Insight: Gambler's Fallacy Refuted

The arcsine law overturns the naive intuition that “fair” games lead to equal sharing of the lead. A player who is behind for the first 99% of a game is *not* especially likely to “recover” — in fact, being behind for the entire game is a completely typical outcome. This is the *gambler's fallacy* refuted: past losses do not make future wins more likely. The game has no memory.

Connection to randomness testing. The arcsine law is not merely a theoretical curiosity. Later in the book (Section 2.5), we will use the arcsine law to construct a *statistical test* for the quality of a pseudorandom number generator: if the generator produces numbers that are truly random, then the L^+ statistic should follow the arcsine distribution. A significant deviation from the arcsine law signals a defect in the generator.

1.2.4 Python: Simulating the SSRW and the LIL Envelope

The following code simulates a single SSRW trajectory and overlays the LIL envelope $\pm\sqrt{2n\log\log n}$. The key implementation step is converting uniform integers in $\{0, 1\}$ to steps in $\{-1, +1\}$ via the formula $X_j = 1 - 2B_j$ where $B_j \in \{0, 1\}$: when $B_j = 0$ we get $X_j = 1$ (heads), and when $B_j = 1$ we get $X_j = -1$ (tails). Then `np.cumsum` (cumulative sum) computes all the partial sums at once.

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.random import default_rng

rng = default_rng(seed=31415)
N = 1000                                # walk has 2N steps

steps = 1 - 2 * rng.integers(0, 2, size=2*N) # {-1, +1}
S = np.concatenate([[0], steps.cumsum()])    # S_0=0, S_1, ..., S_{2N}

# LIL envelope: +/- sqrt(2 * n * log(log(n)))
# We start from n=2 to avoid log(log(1))=log(0) which is -infinity
n_arr = np.arange(2, 2*N + 1)
envelope = np.sqrt(2 * n_arr * np.log(np.log(n_arr)))

fig, ax = plt.subplots(figsize=(10, 3.5))
ax.plot(range(2*N + 1), S, lw=0.8, color='steelblue', label='$S_n$')
ax.plot(n_arr, envelope, 'b--', lw=1.8, alpha=0.7,
        label=r'LIL:  $\pm\sqrt{2n\log\log n}$ ')
ax.plot(n_arr, -envelope, 'b--', lw=1.8, alpha=0.7)
ax.axhline(0, color='k', lw=0.5, ls='--')
```

1.2.5 Python: Verifying the Arcsine Law

To verify the arcsine law statistically, we simulate $R = 10,000$ independent walks of $2n = 1000$ steps each, compute $L_{2n}^+/(2n)$ for each walk, and compare the histogram to the theoretical arcsine density. Vectorising the simulation over all R replications simultaneously using NumPy's array operations makes this fast enough to run interactively.

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.random import default_rng

def fraction_above_zero(n=500, R=10_000, seed=0):
    """Return array of  $L_{2n}^+/(2n)$  for  $R$  independent walks.

    n : half-length (walk has 2n steps)
    R : number of independent replications
    Returns: array of shape (R,), each value in [0,1]
    """
    rng = default_rng(seed)
    # shape: (R, 2n) -- each row is one walk's steps
    steps = 1 - 2 * rng.integers(0, 2, size=(R, 2*n))
    # cumsum along axis=1, prepend column of zeros --> shape (R, 2n+1)
    S = np.hstack([np.zeros((R, 1)), steps.cumsum(axis=1)])
    # Step j is counted if  $S_{j-1} \geq 0$  OR  $S_j \geq 0$  (at least one endpoint  $\geq 0$ )
    above = (S[:, :-1] >= 0) | (S[:, 1:] >= 0)
    return above.sum(axis=1) / (2*n) # shape (R,), each in [0,1]

fracs = fraction_above_zero(n=500, R=10_000)
```

1.3.1 The TSP: Problem Statement and Why It Is Hard I

The Travelling Salesman Problem (TSP) is a paradigmatic hard combinatorial optimisation problem. It arises in logistics (planning delivery routes), circuit board design (minimising wire length), genome sequencing (ordering DNA fragments), and many other practical domains.

The problem. A salesman must visit n cities exactly once and return to the starting city. The *distance* (or cost) of travelling from city i to city j is given by the entry $\mathbf{M}_{ij}$ ($\mathbf{M}$ subscript ij) in an $n \times n$ matrix $\mathbf{M}$ (bold $\mathbf{M}$, an array of distances). The entry is $\mathbf{M}_{ij} = \infty$ (infinity) if there is no direct route from i to j . The goal is to find the *shortest closed tour* (a route that starts and ends at the same city and visits each city exactly once).

Mathematically, a *tour* is a permutation σ (sigma, a re-ordering) of the cities $\{1, \dots, n\}$. The symbol $\sigma(k)$ (sigma of k) means the k -th city visited. The total length of the tour is:

$$l(\sigma) = \sum_{k=1}^{n-1} \mathbf{M}_{\sigma(k), \sigma(k+1)} + \mathbf{M}_{\sigma(n), \sigma(1)}.$$

The first sum adds up the distances of all legs except the last, and the final term $\mathbf{M}_{\sigma(n), \sigma(1)}$ closes the loop by returning to the starting city.

1.3.1 The TSP: Problem Statement and Why It Is Hard II

Optimisation Problem

Find the optimal permutation (the shortest tour):

$$\sigma^* = \arg \min_{\sigma \in S_n} l(\sigma),$$

where S_n (S subscript n) denotes the set of all $n!$ permutations of n cities, and “arg min” means “the permutation that achieves the minimum value of $l(\sigma)$ ”.

1.3.1 The TSP: Problem Statement and Why It Is Hard III

Why is the TSP computationally hard?

The search space has $n!$ (n factorial) elements. The factorial function grows faster than any exponential:

n cities	$n!$ permutations	Time at 10^9 /sec
10	$3,628,800 \approx 3.6 \times 10^6$	0.004 seconds
15	$\approx 1.3 \times 10^{12}$	21 minutes
20	$\approx 2.4 \times 10^{18}$	76 years
50	$\approx 3.0 \times 10^{64}$	longer than the age of the universe

At 10^9 (one billion) permutation evaluations per second, checking all $50!$ tours would require approximately 10^{55} years — the current age of the universe is only about 1.4×10^{10} years. Exhaustive search is completely infeasible for $n \geq 20$.

The TSP belongs to the class of **NP-complete** problems: no algorithm is known that solves all instances in polynomial time (time proportional to n^k for some fixed k), and most computer scientists believe no such algorithm exists. Exact algorithms are practical only for $n \lesssim 50$ (using clever branch-and-bound methods), and even then they can be slow.

1.3.1 The TSP: Problem Statement and Why It Is Hard IV

Monte Carlo heuristics for the TSP. Since exact optimisation is infeasible for large n , we use *randomised approximation algorithms* that find near-optimal (but not necessarily optimal) solutions quickly.

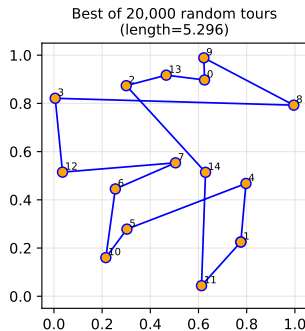
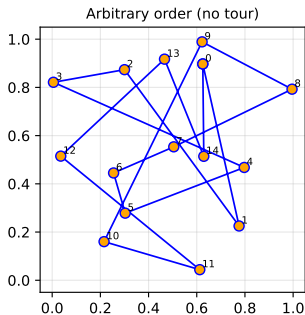
Random restart. Sample R (integer R) random permutations uniformly from S_n , evaluate $l(\sigma)$ for each, and return the best. This is the simplest Monte Carlo approach. For $n = 15$ cities and $R = 20,000$ trials, random search typically finds a tour that is 30–50% longer than the optimal. The figure below shows the result.

Simulated Annealing (Ch. 7). Start with a random tour. At each step, propose a small local modification called a *2-opt move*: reverse a segment of the tour (remove two edges and reconnect the endpoints differently). Always accept improvements. Accept worsening moves with probability $\exp(-\Delta l/T)$ (exp of minus Delta- l divided by T), where Δl (Delta l) is the increase in tour length and T (T) is a “temperature” parameter that decreases over time. The probabilistic acceptance of worsening moves allows escape from local optima. This approach, introduced by Kirkpatrick, Gelatt and Vecchi (1983), produces near-optimal tours far more reliably than random restart.

1.3.1 The TSP: Problem Statement and Why It Is Hard V

Cross-Entropy Method (Chs. 5, 7). Maintain a probability distribution over tours, sample batches of tours, keep the best fraction (the “elite sample”), and update the distribution to fit the elite tours. Iterating drives the distribution toward optimal tours.

Travelling Salesman Problem – random search heuristic



1.3.1 The TSP: Problem Statement and Why It Is Hard VI

Left: 15 random cities connected in an arbitrary random order (no optimisation; many edge crossings visible). Right: best tour found by random search over $R = 20,000$ random permutations. The absence of crossings is a necessary condition for an optimal tour.

1.3.2 Python: Random Tour Search for the TSP

The following implementation evaluates $R = 20,000$ random tours of $n = 15$ cities and keeps the shortest one. The function `tour_length` uses `np.roll` (“roll” or shift the array by one position) to implement the cyclic distance computation efficiently without an explicit loop.

```
import numpy as np
from numpy.random import default_rng

def tour_length(cities, perm):
    """Compute the total Euclidean length of a closed tour.

    cities : shape (n, 2) -- the (x, y) coordinates of each city
    perm    : permutation of {0, ..., n-1} -- the order to visit cities
    Returns: the total distance of the tour (a single float)
    """
    # shifted[k] = perm[k+1 mod n], so it gives the NEXT city after perm[k]
    # This creates the "closing the loop" effect automatically
    shifted = np.roll(perm, -1)
    # Compute (x, y) differences for each leg of the tour: shape (n, 2)
    diffs    = cities[perm] - cities[shifted]
    # Euclidean distance of each leg: sqrt(dx^2 + dy^2), then sum all legs
    return np.linalg.norm(diffs, axis=1).sum()

def random_tour_search(cities, R=20_000, seed=42):
    """Return (best_tour, best_length) from R random permutations.

    R      : number of random tours to try (more is better but slower)
    seed   : random seed for reproducibility
```

1.4.1 The Monte Carlo Integration Framework I

One of the most important applications of Monte Carlo methods is numerical estimation of integrals. This section develops the basic theory and shows why MC is *dimension-independent* in its convergence rate — a remarkable property that grid-based quadrature methods do not share.

The integration problem. We want to compute:

$$I = \int_{[0,1]^d} k(\mathbf{u}) d\mathbf{u},$$

where $k : [0, 1]^d \rightarrow \mathbb{R}$ (k , a function from the d -dimensional unit cube to the real numbers) is an integrable function, $[0, 1]^d$ (pronounced “the d -dimensional unit hypercube”) is the set of all d -dimensional vectors with each coordinate between 0 and 1, and $\mathbf{u}$ (bold u) is a d -dimensional variable of integration. This formulation covers a large class of problems because any integral over a bounded domain can be transformed to this form by a change of variables.

The probabilistic interpretation. If $\mathbf{U}$ (bold U) is a random vector *uniformly distributed* on $[0, 1]^d$ (meaning each coordinate is independently and uniformly distributed on $[0, 1)$), then:

$$I = \mathbb{E}[k(\mathbf{U})].$$

1.4.1 The Monte Carlo Integration Framework II

The integral equals the *expected value* (average) of $k(\mathbf{U})$ when $\mathbf{U}$ is uniform. This is the fundamental connection between integration and probability that makes Monte Carlo work.

Monte Carlo Estimator (formula 1.2.2 in the book)

Let $\mathbf{U}_1, \mathbf{U}_2, \dots, \mathbf{U}_n$ be i.i.d. (independent and identically distributed) uniform random vectors on $[0, 1]^d$. Define the **Monte Carlo estimator**:

$$\hat{Y}_n = \frac{1}{n} \sum_{i=1}^n k(\mathbf{U}_i). \quad (1.2.2)$$

Here $\hat{Y}_n$ (Y-hat sub n; the hat symbol $\hat{}$ means “estimate”) is the sample average of n independent evaluations of k . By the Strong Law of Large Numbers (SLLN), $\hat{Y}_n \rightarrow I$ almost surely (with probability 1) as $n \rightarrow \infty$. The estimator is **unbiased**: $\mathbb{E}[\hat{Y}_n] = I$ for every n .

1.4.1 The Monte Carlo Integration Framework III

Variance and Error Rate of the MC Estimator

Since the $\mathbf{U}_i$ are i.i.d., the variance (the square of the spread around the mean) of $\hat{Y}_n$ is:

$$\text{Var}(\hat{Y}_n) = \frac{\sigma^2}{n}, \quad \text{where} \quad \sigma^2 = \text{Var}(k(\mathbf{U})) = \int_{[0,1]^d} k(\mathbf{u})^2 d\mathbf{u} - I^2.$$

Here σ^2 (sigma-squared) is the variance of a single evaluation $k(\mathbf{U})$, and n (n) is the number of samples. The standard deviation σ (sigma) measures the typical spread of a single evaluation around the mean I . By the Central Limit Theorem (CLT), for large n :

$$\sqrt{n}(\hat{Y}_n - I) \xrightarrow{d} \mathcal{N}(0, \sigma^2),$$

meaning $\sqrt{n}$ (square root of n) times the error converges *in distribution* ($\xrightarrow{d}$) to a normal distribution with mean 0 and variance σ^2 . Consequently:

$$\text{RMSE}(\hat{Y}_n) = \sqrt{\mathbb{E}[(\hat{Y}_n - I)^2]} = \frac{\sigma}{\sqrt{n}} = O(n^{-1/2}).$$

Here RMSE (root mean squared error) is the standard measure of estimation accuracy, and $O(n^{-1/2})$

1.4.2 Python: Estimating π with Monte Carlo

The classic demonstration of MC integration is estimating $\pi \approx 3.14159$ (pi, the ratio of a circle's circumference to its diameter). A quarter of the unit disk — the set $\{(x, y) : x^2 + y^2 \leq 1, x \geq 0, y \geq 0\}$ — has area $\pi/4$. By throwing random points uniformly in the unit square $[0, 1]^2$ and counting how many land inside the quarter disk, we estimate the area $\pi/4$, and hence π .

Formally, we set $k(u_1, u_2) = 4 \cdot \mathbf{1}[u_1^2 + u_2^2 \leq 1]$ (four times the indicator function that is 1 if inside the quarter circle and 0 otherwise), so that $I = \mathbb{E}[k(\mathbf{U})] = 4 \cdot (\pi/4) = \pi$. The factor of 4 converts the area of the quarter disk to π .

```
import numpy as np
from numpy.random import default_rng

def estimate_pi(n, seed=0):
    """Estimate pi by the quarter-circle Monte Carlo method.

    n      : number of random points to use
    seed   : random seed for reproducibility
    Returns: estimate of pi (a float near 3.14159)
    """
    rng     = default_rng(seed)
    U       = rng.random((n, 2))           # n points, each (x,y) in [0,1)^2
    inside  = (U**2).sum(axis=1) <= 1.0     # True if x^2 + y^2 <= 1
    return 4.0 * inside.mean()             # 4 * (fraction inside)
```

1.5.1 The Idea Behind Quasi-Monte Carlo I

The MC error rate $O(n^{-1/2})$ is a direct consequence of the *randomness* of the sample points: random points inevitably form clusters and leave gaps, resulting in discrepancy $D = O(n^{-1/2})$. This raises a natural question: can we do better by replacing random points with *carefully constructed deterministic* points that fill $[0, 1]^d$ more uniformly?

The answer is yes — this is the idea of **Quasi-Monte Carlo (QMC)** methods.

Low-Discrepancy Sequences

A sequence $\mathbf{x}_1, \mathbf{x}_2, \dots \in [0, 1]^d$ is called a *low-discrepancy sequence* (or *quasirandom sequence*) if its star-discrepancy satisfies:

$$D(\mathbf{x}_1, \dots, \mathbf{x}_n; \mathcal{B}^*) = O\left(\frac{(\log n)^d}{n}\right).$$

Here $(\log n)^d$ (log-n to the power d) is a dimension-dependent logarithmic factor; n (n) is the number of points. For fixed d , this is much smaller than the MC rate of $O(n^{-1/2})$.

1.5.1 The Idea Behind Quasi-Monte Carlo II

For a low-discrepancy sequence, the QMC estimator $\hat{I}_n^{\text{QMC}} = \frac{1}{n} \sum_{i=1}^n k(\mathbf{x}_i)$ converges to I at rate $O(n^{-(1-\varepsilon)})$ for any $\varepsilon > 0$ (any small positive number), under mild smoothness conditions on k . For smooth integrands in moderate dimensions, this is dramatically faster than the $O(n^{-1/2})$ MC rate.

1.5.1 The Idea Behind Quasi-Monte Carlo III

Koksma–Hlawka Inequality (Deterministic Error Bound)

If $V_{\text{HK}}(k)$ (V-HK of k , the Hardy–Krause variation of k) measures the total variation of k over $[0, 1]^d$, then:

$$\left| \hat{I}_n^{\text{QMC}} - I \right| \leq V_{\text{HK}}(k) \cdot D(\mathbf{x}_1, \dots, \mathbf{x}_n).$$

This is a *deterministic* (non-random) error bound. If the discrepancy D is small (points fill space evenly) *and* k is smooth (bounded variation $V_{\text{HK}}(k) < \infty$), then the QMC error is provably small. No such deterministic bound exists for random MC — for MC we can only give *probabilistic* error guarantees. The best known low-discrepancy sequences satisfy:

$$D(\mathbf{x}_1, \dots, \mathbf{x}_n) \leq C_d \cdot \frac{(\log n)^d}{n} + O\left(\frac{(\log n)^{d-1}}{n}\right),$$

where C_d (C subscript d) is a constant that depends on the dimension d but not on n . For small d and large n , this bound is much smaller than the MC discrepancy $O(n^{-1/2})$, explaining QMC's faster convergence. However, C_d grows rapidly with d , so for very large dimensions ($d \gtrsim 50$), QMC's advantage disappears.

1.5.1 The Idea Behind Quasi-Monte Carlo IV

Popular low-discrepancy sequences. All are available in Python via the `scipy.stats.qmc` module (SciPy version 1.7 and later):

Halton sequence. Generalises the Van der Corput sequence to d dimensions by using different prime bases $p_1, p_2, \dots, p_d$ per dimension. For example, in 2 dimensions: dimension 1 uses base 2 (expressing fractions in binary), dimension 2 uses base 3 (ternary). Simple to understand and implement. For large dimensions, correlation between prime-based sequences can appear.

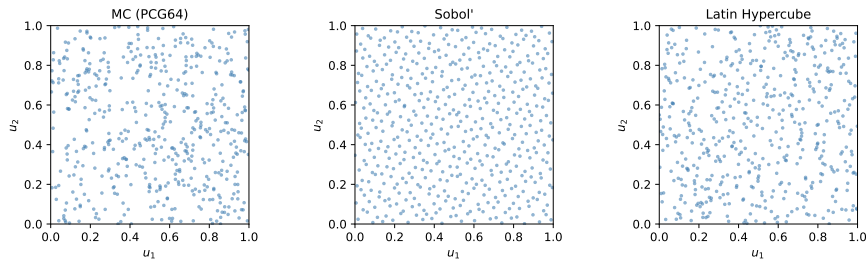
Sobol' sequence. Constructed using binary arithmetic (Gray codes). Excellent uniformity for $d \geq 2$. Best results when $n = 2^k$ for integer k . Widely used in financial mathematics (option pricing).

Faure sequence. Based on permuted base- p representations. Has rigorous discrepancy bounds with explicit constants C_d .

Latin Hypercube Sampling (LHS). Divides each dimension into n equally spaced intervals and ensures exactly one sample falls in each row and each column of the resulting $n \times d$ grid. Guarantees perfect marginal uniformity but is not a strictly low-discrepancy sequence in the full d -dimensional sense.

1.5.1 The Idea Behind Quasi-Monte Carlo V

$n = 512$ points in $[0, 1]^2$: random vs low-discrepancy



$n = 512$ points in $[0, 1]^2$. Left: MC (PCG64) — visible clusters and empty gaps. Centre: Sobol' — noticeably more regular, no obvious gaps. Right: Latin Hypercube — perfect marginal uniformity.

1.5.2 The Golden-Ratio and Kronecker Sequences I

Besides the Halton and Sobol' sequences, there is a beautiful and elegant class of low-discrepancy sequences based on *irrational rotations* of the unit interval.

The Golden-Ratio (Steinhaus) Sequence in 1D

Define the **golden ratio**:

$$\alpha = \frac{\sqrt{5} - 1}{2} \approx 0.6180\dots,$$

which is the positive root of the equation $z^2 + z - 1 = 0$. The golden-ratio sequence is defined by taking the *fractional parts*:

$$x_n = \{n\alpha\} = n\alpha - \lfloor n\alpha \rfloor, \quad n = 1, 2, 3, \dots$$

Here $\{x\}$ (braces around x) denotes the *fractional part* of x — for example, $\{3.7\} = 0.7$ and $\{5.2\} = 0.2$ — and $\lfloor x \rfloor$ (floor of x) is the largest integer not exceeding x . The resulting sequence (x_n) is a low-discrepancy sequence on $[0, 1)$.

1.5.2 The Golden-Ratio and Kronecker Sequences II

Why is the golden ratio special? The golden ratio has the *continued fraction expansion*:

$$\alpha = \frac{1}{1 + \frac{1}{1 + \frac{1}{1 + \dots}}}$$

where all “partial quotients” (the numbers in the denominators) equal 1. A number is *easily approximated by rationals* if it has large partial quotients; such numbers produce sequences that cluster near rational fractions and have high discrepancy. The golden ratio, with all partial quotients equal to 1, is the “*most irrational*” number — the one hardest to approximate by rationals — and therefore produces the most evenly spread sequence.

1.5.2 The Golden-Ratio and Kronecker Sequences III

Kronecker Sequences (Generalisation)

More generally, $x_n = \{n\beta\}$ (fractional part of $n\beta$) is a low-discrepancy sequence whenever β (beta) has a continued fraction expansion with *bounded partial quotients*:

$$\beta = \frac{1}{b_1 + \frac{1}{b_2 + \frac{1}{b_3 + \dots}}},$$

where $b_j \leq M$ (b -subscript- j is at most some constant M) for all $j = 1, 2, 3, \dots$. The golden ratio satisfies this with $b_j = 1$ for all j , giving the tightest possible bound.

Multidimensional Kronecker sequences. The natural extension to d dimensions is the sequence $(\{n\beta_1\}, \dots, \{n\beta_d\})$ (taking the fractional parts of n times d different irrational numbers, one per dimension). Whether such sequences are provably low-discrepancy in $d \geq 2$ dimensions is an *open*

1.5.2 The Golden-Ratio and Kronecker Sequences IV

problem in number theory. However, extensive numerical experiments suggest they behave well in practice.

Critical Warning: Do Not Slice a 1D Sequence into d -Vectors

It is *incorrect* to take a 1-D low-discrepancy sequence $x_1, x_2, \dots$ and group consecutive values into d -dimensional vectors: $(x_1, \dots, x_d), (x_{d+1}, \dots, x_{2d}), \dots$. The resulting sequence of d -vectors does *not* inherit the low-discrepancy property in d dimensions — the 2D points will lie along diagonal lines rather than filling the square uniformly. Each dimension must be constructed simultaneously, as the `scipy.stats.qmc` library does correctly.

1.5.3 Python: QMC Sequences with `scipy.stats.qmc`

The `scipy.stats.qmc` module (SciPy version 1.7 and later) provides implementations of all major low-discrepancy sequences. The interface is uniform: each class has a `random(n)` method that returns an $n \times d$ NumPy array of quasi-random points in $[0, 1)^d$. Setting `scramble=True` adds a random scrambling that preserves the low-discrepancy property while making the estimator unbiased (its expected value equals the true integral).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats.qmc import Sobol, Halton, LatinHypercube
from numpy.random import default_rng

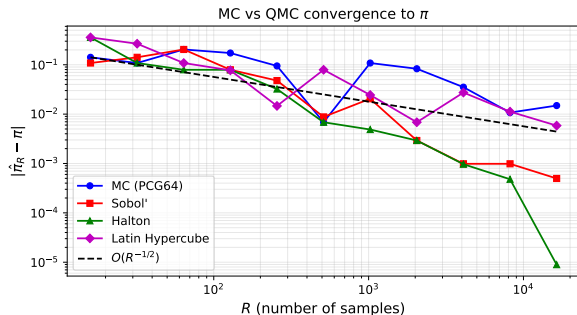
def estimate_pi_from(U):
    """Estimate pi from an n-by-2 array U of points in [0,1)^2."""
    return 4.0 * ((U**2).sum(axis=1) <= 1.0).mean()

# Compare convergence over a range of sample sizes (powers of 2)
# Sobol' works best for n = 2^k, so we use powers of 2 throughout
sample_sizes = [2**k for k in range(4, 15)] # 16, 32, 64, ..., 16384
mc_err, sob_err, hal_err, lhs_err = [], [], [], []

for n in sample_sizes:
    rng = default_rng(seed=0)
    # MC: pseudorandom points (independent uniform)
    mc_err.append(abs(estimate_pi_from(rng.random((n, 2))) - np.pi))
    # Sobol' (scrambled for unbiasedness)
    sob_err.append(abs(estimate_pi_from(
```

1.6.1 Example 1: Estimating π — Where QMC Wins I

The figure below shows the absolute error $|\hat{\pi}_n - \pi|$ (the vertical axis, on a logarithmic scale) as a function of sample size n (horizontal axis, also logarithmic). On a log-log plot, a rate of $O(n^{-1/2})$ appears as a straight line with slope $-1/2$, and a rate of $O(n^{-1})$ appears as a line with slope -1 .



Reading the plot.

- The black dashed line is the reference $O(n^{-1/2})$ slope. The blue MC curve follows this slope closely.
- Sobol' (red) and Halton (green) lie well *below* the MC line, achieving much smaller errors at the same n .
- For large n , the QMC slopes are steeper than $-1/2$, approaching -1 for this smooth integrand.
- At $n = 2^{14} = 16,384$: Sobol' achieves error $\approx 2 \times 10^{-5}$, while MC achieves only $\approx 5 \times 10^{-4}$ — a factor of **25 improvement**.

1.6.1 Example 1: Estimating π — Where QMC Wins II

Why does QMC work so well for estimating π ?

The integrand is $k(u_1, u_2) = 4 \cdot \mathbf{1}[u_1^2 + u_2^2 \leq 1]$ (four times the indicator of the quarter-disk). This function is smooth everywhere except along the quarter-circle boundary, where it has a jump discontinuity. The Hardy–Krause variation $V_{\text{HK}}(k)$ is finite (because the boundary is a smooth curve), so the Koksma–Hlawka inequality applies with $D_n = O((\log n)^2/n)$, giving convergence approaching $O(n^{-1})$.

However, the indicator's jump means this is a borderline case: for integrands with jump discontinuities *aligned with the coordinate axes* (e.g., $k(u_1, u_2) = \mathbf{1}[u_1 \leq 0.3]$), the Hardy–Krause variation becomes infinite and QMC may not converge faster than MC.

When QMC Wins and When It Does Not

QMC is effective for *smooth integrands in moderate dimensions* ($d \lesssim 10\text{--}20$). For non-smooth functions, very high dimensions ($d \gtrsim 50$), or — crucially — *simulation tasks where the number of random numbers consumed per trial varies*, QMC can perform worse than standard MC. The next section illustrates this failure.

1.6.2 Example 2: A 2-D Game — Where QMC Fails I

The second comparison example illustrates a setting where quasi-random numbers are *harmful* — they introduce systematic bias and cause the estimator to converge to the *wrong value*.

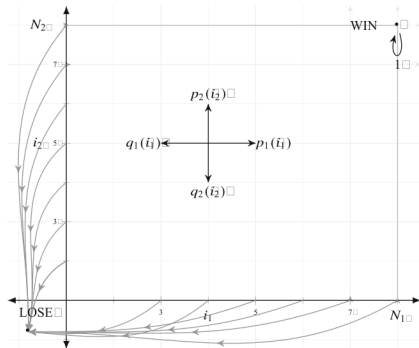
The 2-D game. Consider a grid with $N_1 \times N_2$ states (an N_1 -by- N_2 rectangular grid of positions). We start at state (i_1, i_2) (position i_1 along the first axis, i_2 along the second) and move stochastically (randomly) at each step. The game ends in two ways:

- If any coordinate hits 0: we **LOSE** (bottom-left corner).
- If both coordinates reach their maxima N_1 and N_2 respectively: we **WIN** (top-right corner).

From state (i_1, i_2) , the four possible transitions are:

- $(i_1, i_2) \rightarrow (i_1 + 1, i_2)$ with probability $p_1(i_1)$ (step right),
- $(i_1, i_2) \rightarrow (i_1 - 1, i_2)$ with probability $q_1(i_1)$ (step left),
- $(i_1, i_2) \rightarrow (i_1, i_2 + 1)$ with probability $p_2(i_2)$ (step up),
- $(i_1, i_2) \rightarrow (i_1, i_2 - 1)$ with probability $q_2(i_2)$ (step down),
- and the state remains unchanged with the remaining probability.

1.6.2 Example 2: A 2-D Game — Where QMC Fails II



State space of the 2-D game. LOSE is the bottom-left boundary; WIN is the top-right corner. Arrows show the four possible moves from a generic interior state (i_1, i_2) .

1.6.2 Example 2: A 2-D Game — Where QMC Fails III

Exact Winning Probability (Formula 1.4.1)

Let $\rho(i_1, i_2) = \mathbb{P}(\text{WIN} \mid \text{start at } (i_1, i_2))$ (rho, the winning probability given we start at position (i_1, i_2)). A closed-form formula is:

$$\rho(i_1, i_2) = \frac{\prod_{j=1}^2 \left(\sum_{n_j=1}^{i_j} \prod_{r=1}^{n_j-1} \frac{q_j(r)}{p_j(r)} \right)}{\prod_{j=1}^2 \left(\sum_{n_j=1}^{N_j} \prod_{r=1}^{n_j-1} \frac{q_j(r)}{p_j(r)} \right)}, \quad (1.4.1)$$

where $\prod$ (Pi, meaning “product”) multiplies terms together, $\sum$ (Sigma, meaning “sum”) adds terms, $q_j(r)/p_j(r)$ is the ratio of the downward to upward transition probability at position r , and empty products (when $n_j = 1$) equal 1 by convention.

1.6.2 Example 2: A 2-D Game — Where QMC Fails IV

The symmetric special case. Set $N_1 = N_2 = 4$ and $p_j(i_j) = q_j(i_j) = 1/4$ for all $i_j \in \{1, 2, 3\}$. Then all ratios $q_j(r)/p_j(r) = 1$, so all products equal 1 and all sums simplify. The formula reduces to $\rho(i_1, i_2) = (i_1/N_1)(i_2/N_2)$. Starting at $(i_1, i_2) = (2, 3)$:

$$\rho(2, 3) = \frac{2}{4} \cdot \frac{3}{4} = \frac{3}{8} = 0.375.$$

This is the *exact* winning probability that our simulation should estimate.

1.6.2 Example 2: A 2-D Game — Where QMC Fails V

Simulation algorithm. To estimate $\rho(2, 3)$, we play the game R times starting from $(2, 3)$ and record $Y_j \in \{0, 1\}$ (0 for LOSE, 1 for WIN) for each game. The Monte Carlo estimator is:

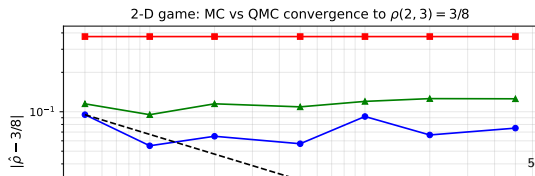
$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R Y_j,$$

the fraction of games that end in WIN. Each step of the game uses a pair of uniform numbers $(U_1, U_2) \in [0, 1)^2$ to decide the move.

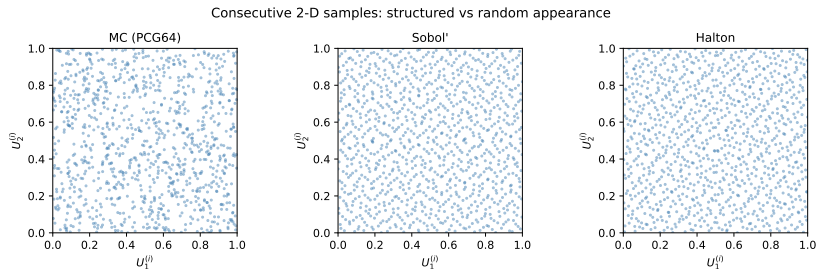
Why QMC fails here. Each game requires a *variable number* of (U_1, U_2) pairs from the random stream, depending on how long the game lasts. Game j 's duration is random (determined by previous outcomes), so after game j the stream is at a random position. The structured correlations of a QMC sequence (its points lie along regular patterns) are destroyed when the stream is accessed at random positions. This causes *systematic bias*.

Results (book Table 1.2, $R = 5000$ games)

Algorithm	$\hat{Y}_{5000}$	Avg. steps
BCC64 (MC)	0.3760	8.41



1.6.2 Example 2: A 2-D Game — Where QMC Fails VI



Scatter of consecutive pairs $(U_1^{(i)}, U_2^{(i)})$ for $n = 1024$. MC (left): no visible structure — the pairs look random. Sobol' (centre) and Halton (right): clear linear patterns. When a game's stream position depends on previous random outcomes, these structured patterns create correlations between successive games and cause systematic bias in the estimator.

1.6.2 Example 2: A 2-D Game — Where QMC Fails VII

Practical Rule

Use QMC for **deterministic quadrature** (a fixed, predetermined set of function evaluations at points chosen before the computation begins). Do **not** use QMC when the number of random numbers consumed per trial depends on the random outcome, as in any Markov chain or sequential simulation. Standard MC is the safe and correct choice for all such simulation tasks.

1.6.3 Python: 2-D Game Simulation (MC Version)

The following code simulates the symmetric 2-D game ($N_1 = N_2 = 4$, $p_j = q_j = 1/4$) using standard pseudo-random numbers, which correctly converges to $\rho(2, 3) = 0.375$.

```
import numpy as np
from numpy.random import default_rng

N1, N2 = 4, 4          # grid dimensions
p = q = 0.25          # symmetric:  $p_j(i_j) = q_j(i_j) = 1/4$  for all  $i_j$ 

def play_one_game(rng, start=(2, 3), max_steps=10_000):
    """Play one 2-D game using consecutive (U1, U2) pairs from rng.

    At each step:
        U1 in [0, 0.5) -> change coordinate 1
        U1 in [0.5, 1) -> change coordinate 2
        U2 < 0.5       -> move UP (+1) in the chosen coordinate
        U2 >= 0.5      -> move DOWN (-1) in the chosen coordinate

    Returns 1 if WIN (both coords reach N), 0 if LOSE (any coord hits 0).
    """
    i1, i2 = start
    for _ in range(max_steps):
        U1, U2 = rng.random(), rng.random()
        if U1 < 2*p:          # change coordinate 1 (prob =  $p + q = 0.5$ )
            i1 += 1 if U2 < 0.5 else -1
        else:                # change coordinate 2 (prob =  $p + q = 0.5$ )
            i2 += 1 if U2 < 0.5 else -1
```

1.7 Bibliographical Notes I

Origins of Monte Carlo. The modern Monte Carlo method was developed at the Los Alamos National Laboratory in the late 1940s. Stanislaw Ulam conceived the basic idea while recovering from illness and playing solitaire: he realised that estimating probabilities by repeated random sampling could solve neutron diffusion equations that were intractable analytically. John von Neumann joined the effort and developed the theoretical foundations; Nicholas Metropolis implemented the method on early computers and coined the name “Monte Carlo.” The first published paper naming the method is Metropolis & Ulam (1949), *The Monte Carlo Method*, *Journal of the American Statistical Association*, **44**(247):335–341.

Pseudorandom number generation. The foundational reference is D. E. Knuth [75], *The Art of Computer Programming*, Vol. 2, Chapter 3, which provides a comprehensive treatment of congruential generators and statistical tests for randomness. L’Ecuyer [82] surveys modern PRNGs with long periods and good statistical properties. Niederreiter [101] covers both PRNGs and quasi-random sequences. The PCG64 generator used in NumPy was proposed by M. E. O’Neill (2014): *PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation*, Preprint, Harvey Mudd College. PCG64 has period 2^{128} , excellent statistical quality, and is the recommended choice for Monte Carlo simulation in Python.

1.7 Bibliographical Notes II

Random walks and limit theorems. The arcsine law was established by W. Feller [52], *An Introduction to Probability Theory and Its Applications*, Vol. I, Chapters III and XIV. This book remains the most accessible and complete treatment of classical random walk theory. The Law of the Iterated Logarithm is due to A. Khintchine (1924) and A. N. Kolmogorov (1929); a modern probabilistic treatment is in Gut [100]. R. Durrett [43], *Probability: Theory and Examples* (4th edition, Cambridge), provides a rigorous treatment of random walk limit theorems using martingales and Brownian motion. The connection between the arcsine law and randomness testing of PRNGs is discussed in L'Ecuyer & Simard [82].

Quasi-Monte Carlo methods. The foundational monograph is H. Niederreiter [101], *Random Number Generation and Quasi-Monte Carlo Methods* (SIAM, 1992), which establishes the theory of low-discrepancy sequences, the Koksma–Hlawka inequality, and construction of the Halton, Faure, and Sobol' sequences. The Halton sequence was introduced in J. H. Halton [60] (1960); the Sobol' sequence in I. M. Sobol' [119] (1967); the Faure sequence in H. Faure [50] (1982). For scrambling and randomisation of QMC sequences (needed for unbiasedness), see S. Tezuka [124]. For financial applications of QMC, see P. Glasserman [55], *Monte Carlo Methods in Financial Engineering* (Springer,

1.7 Bibliographical Notes III

2003). In Python, the `scipy.stats.qmc` module (SciPy ≥ 1.7) provides Sobol, Halton, and LatinHypercube with optional scrambling.

Travelling Salesman Problem. The TSP is studied exhaustively in D. L. Applegate, R. E. Bixby, V. Chvátal & W. J. Cook [4], *The Traveling Salesman Problem: A Computational Study* (Princeton University Press, 2006). Simulated annealing for combinatorial optimisation was introduced by S. Kirkpatrick, C. D. Gelatt & M. P. Vecchi [74] (1983), *Optimization by Simulated Annealing*, *Science*, **220**(4598):671–680. The two-player game used to illustrate QMC failure is from Lefebvre, L'Ecuyer & Simard [129]; the exact formula for the winning probability is derived there.

Chapter 1 Summary I

Chapter 1 introduces Monte Carlo through three concrete motivating examples — the simple symmetric random walk, the Travelling Salesman Problem, and numerical integration — and contrasts standard Monte Carlo with quasi-Monte Carlo for numerical integration.

Chapter 1 Summary II

Core Concepts from Chapter 1

- ❶ **The Monte Carlo estimator:** $\hat{Y}_n = \frac{1}{n} \sum_{i=1}^n k(\mathbf{U}_i)$ is unbiased ($\mathbb{E}[\hat{Y}_n] = I$), consistent (converges to I almost surely by the SLLN), and has $\text{RMSE} = \sigma/\sqrt{n} = O(n^{-1/2})$ that is *independent of the dimension d* of the problem.
- ❷ **Simple symmetric random walk** $S_n = \sum_{j=1}^n X_j$, where each $X_j \in \{-1, +1\}$ with equal probability:
 - The **Law of the Iterated Logarithm** (LIL): the walk fluctuates in the almost-sure envelope $\pm\sqrt{2n \log \log n}$, which grows slightly faster than $\sqrt{n}$.
 - The **Arcsine Law**: the fraction $L_{2n}^+/(2n)$ of time player A is winning has a U-shaped arcsine distribution, not a distribution concentrated near $1/2$ as naive intuition predicts.
- ❸ **Travelling Salesman Problem (TSP):** NP-hard with $n!$ possible tours; Monte Carlo heuristics (random restart, simulated annealing, cross-entropy) find near-optimal tours far more efficiently than exhaustive search.
- ❹ **Discrepancy:** Measures the deviation of a point set from perfect uniformity. For i.i.d. uniform points, $D = O(n^{-1/2})$; for low-discrepancy (quasi-random) sequences, $D = O((\log n)^d/n)$.
- ❺ **Quasi-Monte Carlo (QMC):** Low-discrepancy sequences (Sobol', Halton, golden-ratio sequences) achieve convergence approaching $O(n^{-1})$ for smooth integrands via the Koksma–Hlawka